Multi-Source Transaction Normalization and Exploration

Wilmer Salazar
KODIGO, Data Normalization and Structure. Individual activity
November 2, 2026
Repository: https://github.com/wkatir/genai-dev-critical-evaluation

1. Overview

The project, *txnorm*, takes transaction records exported by four systems that each describe a payment in their own way, converts them to a single model, checks them, and lets a user explore the result from an interactive terminal interface. The four sources are a core banking API, a card payment gateway that reports amounts in cents and dates as Unix timestamps, a set of in-store POS terminals with a Spanish-locale export (`1.234,56`, `dd/mm/yyyy`, local time), and a mobile wallet that nests its data and uses numeric status codes.

Everything that depends on the shape of a source lives in one configuration file, `config/rules.toml`: field locations, number and date formats, time zones, the status mapping and the list of supported currencies. Adding a fifth source means adding a section to that file, not changing the code.

2. Running the project

Python 3.11 or newer is required. From a clone of the repository:

```
pip install -r requirements.txt
pip install -e .
python -m txnorm                    # interactive menu, loads every file in data/
python -m txnorm --summary          # metrics as JSON
python -m txnorm --export output    # normalized JSON and CSV, rejected records, metrics
pytest                              # 40 tests
```

3. Data model

Each valid record becomes a *Transaction* with the fields below. Records are processed into a wider envelope first, which also keeps the raw input, the detected source, the list of issues and a short note per field describing which rule produced its value. Only records without errors are promoted to a Transaction, so code that consumes transactions never has to deal with missing or half-parsed values.

Field	Type	Rule
id	text	`source:source_id`; ids are only unique inside their own source
source_id	text	original id, kept as text so leading zeros survive
amount	Decimal	major units, rounded to the currency's decimals, always greater than zero
currency	text	ISO 4217 code from the supported list
status	enum	completed, pending, failed, cancelled or refunded

Field	Type	Rule
occurred_at	datetime	timezone-aware, stored in UTC
description, customer_id	text	optional

4. Normalization rules

Source detection. A record is first matched against an explicit marker field when the source has one (for example `provider = "paygate"`). Otherwise it is matched by structure: the source whose required keys are all present wins, the most specific signature breaks ties, and a genuine tie is rejected rather than guessed.

Amounts. Amounts are handled as Decimal, never as floating point. Text amounts are read with the decimal separator declared for the source, so `1.234,56` from a POS terminal and `1,234.56` from the bank both become 1234.56. Thousands grouping is checked strictly; `1234.56` arriving from a comma-decimal source is rejected because it cannot be read without guessing. Cents are divided by the currency's precision, and values with more decimals than the currency allows are rounded half-even and flagged.

Currency. ISO codes are accepted in any case, and symbols or names (`€`, `US$`, `euro`) go through an alias table. A source can override an alias: `$` means USD for the bank but MXN for the Mexican terminals.

Status. Each canonical status lists the raw values it accepts ("succeeded", "Aprobada", "settled" and so on). The wallet adds its numeric codes on top of the shared list. The configuration loader refuses a file in which one raw value points to two different statuses.

Dates. Each source lists the formats it may send, tried in order: ISO 8601, Unix seconds or milliseconds, or explicit patterns. Times without an offset are interpreted in the source's own time zone (Mexico City, Bogotá); if a source declares none, UTC is assumed and the record is flagged. All dates are stored in UTC.

5. Validation and invalid records

Invalid records are not deleted. They are moved to a quarantine list together with every problem found and the original input, and they are left out of all totals. The normalizer does not stop at the first problem, so a record with a bad currency and a missing date reports both. Two conditions are warnings instead of errors: an amount that had to be rounded and a time zone that had to be assumed. These records stay valid but are marked in the interface.

Problem	Code	Outcome

Not a JSON object; no source or more than one source matches	`MALFORMED_RECORD`
	`UNKNOWN_SOURCE`
	`AMBIGUOUS_SOURCE`
		rejected

Required field absent or empty; id of the wrong type	`MISSING_FIELD` `INVALID_ID`	rejected

Problem	Code	Outcome
Amount unreadable, zero or negative	`INVALID_AMOUNT` `NON_POSITIVE_AMOUNT`	rejected
Currency or status not in the rules	`UNSUPPORTED_CURRENCY` `UNKNOWN_STATUS`	rejected
Date unreadable, in the future or before 2000	`INVALID_DATE` `DATE_OUT_OF_RANGE`	rejected
Same source and id already accepted	`DUPLICATE_ID`	rejected
Amount rounded; time zone assumed	`AMOUNT_ROUNDED` `TIMEZONE_ASSUMED`	kept, flagged

Negative amounts are rejected because the direction of a payment is already expressed by its status; allowing both would make a negative refund ambiguous. For duplicates, an id is claimed by the first *valid* record that uses it, so a corrected resend of a record that was previously rejected is accepted.

6. Metrics and interface

The dashboard reports the number of records processed, valid and rejected, the number of valid records per status, and totals per currency, both for all valid records and for completed ones only. Amounts in different currencies are never added together, since the project has no source of exchange rates.

The interface is a menu-driven terminal application built with the *rich* library. From it the user can browse transactions page by page, filter by status, currency, source, date range, amount range or free text, and sort the results; the totals under the table follow the active filters. The detail view shows each normalized value next to the rule that produced it and the original record, which is usually the quickest way to understand why a record was accepted or rejected. The quarantine view lists rejected records by issue code, and the results can be exported to JSON and CSV. The interface module only displays data; all logic sits in the core modules.

7. Use of AI

Claude Code was used as a programming partner. It proposed the module structure, wrote first versions of the parsers, the interface and the sample data, and ran the tests after each change. Several of its initial suggestions were changed after review:

- Amounts were first parsed as floats. They were moved to Decimal, because floating point cannot represent most cent values exactly.
- `$` was mapped to USD for every source. Per-source aliases were added, since the POS terminals operate in Mexico.
- The amount parser tried to guess the decimal separator. It now uses the separator declared for the source and rejects anything ambiguous.
- Refunds were modelled as negative amounts. Amounts are now always positive and the status carries the direction.

– Validation stopped at the first error. It now collects every problem in the record.
– The first metrics draft added all amounts into one total. Totals are now kept per currency.

Two interface bugs came up while testing by hand: the `[n]ext` prompt disappeared because the terminal library treated it as markup, and one invalid filter value cancelled the remaining filters. Both were fixed and are covered by tests.

8. Test data and verification

`data/valid_transactions.json` contains 20 clean records, five per source, covering every status and currency. `data/edge_cases.json` contains 23 problematic records; each one carries a note with its expected outcome, and one of the tests checks every note against what the program actually does. Run on both files, the program processes 43 records: 24 are valid (two of them with warnings) and 19 are quarantined. The suite has 40 tests covering the parsers, the business rules, the metrics, the filters, configuration errors and two scripted sessions of the interactive menu.